PropLead CRM - Complete Technical Implementation Guide

System Architecture, Database Design, Backend Implementation, Frontend Integration, Authentication, and Interview Preparation

This guide is based on the actual source code in `D:\full-stack-template`. It does not include real database URLs, passwords, JWT secrets, tokens, or private production credentials. The final browser/UI acceptance inspection was intentionally not performed for this document because the task requested not to do it.

Table of Contents

1. Project Overview
2. Complete Project Structure
3. System Architecture
4. Frontend Implementation
5. API Integration and Next.js Proxy
6. Backend Architecture, File by File
7. Lead Management - Complete CRUD
8. Search, Filtering, and Sorting
9. Database Design
10. Authentication and Security
11. Notes and Comments
12. Dashboard Implementation
13. User Master and Role Management
14. Validation and Error Handling
15. Environment and Local Setup
16. Testing and Verification
17. End-to-End Data Flow Examples
18. Interview Preparation
19. Final Implementation Summary

1. Project Overview

PropLead CRM is a real estate lead management system. It helps a property sales or CRM team capture enquiries, track each lead's budget and location preference, record notes, update status, and monitor conversion progress from one workspace.

The core problem is that real estate enquiries usually arrive from several sources: Website, Referral, Facebook, Google, walk-in enquiries, and calls. Without a small CRM, the team can lose follow-ups, repeat data entry, or fail to see how many enquiries are moving toward closure.

The users are:

- **Admin** - can access dashboard, leads, deletion, and User Master.
- **CRM** - can access the dashboard and lead pipeline, but cannot delete leads or manage users.

Mandatory assignment features are lead capture, listing, search, filters, sorting, detail view, notes, status updates, dashboard metrics, REST APIs, validation, error handling, database integration, clean code, and setup instructions. Additional features implemented beyond the assignment include JWT login/logout, HttpOnly-cookie proxy authentication, role-based access control, User Master, demo seeding, pagination, budget analytics, and chart rendering.

Current status: the implementation contains the main assignment functionality. Recent verification tested the main workflows through the Next API proxy, Express backend, and PostgreSQL. This document clearly marks limitations such as the missing final browser click inspection in this documentation task.

Technology	Where used	Purpose
Next.js App Router	`frontend/app`	Routing, layouts, server components, and API proxy
React	`frontend/components`	Interactive UI components
Tailwind CSS	JSX class names and `globals.css`	Responsive styling
Express.js	`backend/server.js`, routes	REST API server and middleware
PostgreSQL/Supabase	`backend/sql`	Persistent relational database
pg	`backend/src/db/db.js`	PostgreSQL connection pool
JWT	`backend/src/utils/jwt.js`	Signed session token with user ID, role, expiry
HttpOnly Cookie	`frontend/app/api/[...path]/route.js`	Stores JWT away from browser JavaScript
Recharts	`DashboardCharts.jsx`	Dashboard charts
SweetAlert2	lead/user UI actions	Confirmation and feedback dialogs

2. Complete Project Structure

Important source tree:

```
D:\full-stack-template
  README.md
  package.json
  backend/
    server.js
    .env.example
    sql/schema.sql
    sql/functions.sql
    scripts/setup-db.js
    scripts/seed-demo.js
    scripts/create-admin.js
    src/db/db.js
    src/db/routines.js
    src/middlewares/authMiddleware.js
    src/models/Lead.js
    src/routes/authRoutes.js
    src/routes/leadRoutes.js
    src/routes/dashboardRoutes.js
    src/routes/userRoutes.js
    src/controllers/*Controller.js
    src/services/*Service.js
    src/repositories/*Repository.js
    src/utils/*.js
  frontend/
    .env.example
    api/MainApi.js
    utils/apiRequest.js
    lib/auth.js
    lib/leads.js
    app/api/[...path]/route.js
    app/login/page.jsx
    app/(protected)/layout.jsx
    app/(protected)/page.jsx
    app/(protected)/leads/*
    app/(protected)/users/page.jsx
    components/AppShell.jsx
    components/Dashboard/*
    components/Leads/*
    components/UserMaster/*
```

File	What it does	Depends on / called by	Feature
`backend/server.js`	Creates Express app, middleware order, route mounting, not-found and final error handler	Node entry point	All API features
`leadRoutes.js`	Defines REST lead endpoints and compatibility aliases	`server.js`	Lead CRUD, notes
`leadController.js`	Converts HTTP request data into service calls	`leadRoutes.js`	Lead API
`leadService.js`	Business validation, UUID checks, query checks, note checks	controllers	Lead rules
`Lead.js`	Entity validation for lead fields	`leadService.js`	Create/update lead
`leadRepository.js`	Calls PostgreSQL JSON functions	`leadService.js`	Data access
`db/routines.js`	Allowlisted parameterized PostgreSQL function calls	repositories	Security/data access
`schema.sql`	Tables, constraints, indexes, role seed	setup script/database	Database
`functions.sql`	Application database routines and JSON wrappers	repositories	CRUD/dashboard/auth
`authMiddleware.js`	Bearer token verification and role authorization	`server.js`, routes	Security
`app/api/[...path]/route.js`	Next API proxy and cookie-to-Bearer bridge	browser API calls	Auth integration
`apiRequest.js`	Central axios helper	pages/components/proxy	API integration
`AppShell.jsx`	Role-based sidebar/topbar/logout	protected layout	Navigation
`DashboardComp.jsx`	Server-side dashboard data load and metrics UI	dashboard page	Dashboard
`LeadsComp.jsx`	Client-side listing, filters, pagination, actions	leads page	Lead listing
`LeadForm.jsx`	Create/edit form	new/edit pages	Lead capture/update
`LeadControls.jsx`	Note form, status form, delete helper	edit page	Notes/status
`UserMasterClient.jsx`	User list/create UI	users page	User Master

If SQL were placed in controllers, the HTTP layer would know too much about database structure. If validation were only in SQL, the user would get less friendly errors. The actual project separates concerns so each file is easier to explain and change.

3. System Architecture

Actual request flow:

```
Browser UI
  -> frontend/utils/apiRequest.js
  -> frontend/app/api/[...path]/route.js
  -> Express /api/*
  -> route
  -> controller
  -> service
  -> repository
  -> backend/src/db/routines.js
  -> PostgreSQL function
  -> PostgreSQL table
```

Successful create lead flow:

```
LeadForm.submit
  apiRequest("Leads", "post", values)
  POST /api/leads on Next proxy
  proxy reads proplead_token cookie
  proxy forwards Authorization: Bearer <token>
  Express authenticateToken verifies JWT
  leadRoutes authorizeRoles("admin", "CRM")
  leadController.createLead(req.body)
  leadService.createLead creates new Lead(leadData)
  leadRepository.createLead calls crm_json_create_lead
  PostgreSQL inserts into leads
```

Success response shape:

```
{
  "ok": true,
  "statusCode": 201,
  "message": "Lead created",
  "data": { "id": "...", "name": "..." },
  "pagination": null
}
```

Error response shape:

```
{
  "ok": false,
  "statusCode": 400,
  "message": "Invalid email",
  "data": null,
  "pagination": null
}
```

The project uses real layered architecture, but with a practical twist: repositories call PostgreSQL functions, and the database functions return JSON response payloads for success cases. Infrastructure errors still use backend helpers.

4. Frontend Implementation

The frontend uses Next.js App Router. Public login files are under `frontend/app/login`. Authenticated pages are under `frontend/app/(protected)`. The protected layout calls `requireUser()` before rendering `AppShell`.

```
export default async function ProtectedLayout({ children }) {
  const user = await requireUser();
  return <AppShell user={user}>{children}</AppShell>;
}
```

Line by line: `requireUser()` reads the server-side cookie, calls the backend profile endpoint, and redirects to `/login` if no valid user exists. `AppShell` receives the user and renders role-aware navigation.

Main frontend routes:

Route	File	Purpose
`/login`	`app/login/page.jsx`	Public login
`/`	`app/(protected)/page.jsx`	Dashboard
`/leads`	`app/(protected)/leads/page.jsx`	Lead directory
`/leads/new`	`app/(protected)/leads/new/page.jsx`	Create lead
`/leads/[id]`	`app/(protected)/leads/[id]/page.jsx`	Lead detail and notes
`/leads/[id]/edit`	`app/(protected)/leads/[id]/edit/page.jsx`	Edit lead and add note
`/users`	`app/(protected)/users/page.jsx`	Admin User Master

`LeadsComp.jsx` is a client component because it uses `useState`, `useEffect`, live filtering, pagination reloads, and delete callbacks. It sends query parameters through `apiRequest("Leads", "get", params)`.

Create/edit form excerpt:

```
const result = await apiRequest(
  "Leads",
  lead ? "put" : "post",
  values,
  {},
  lead ? `/${lead.id}` : "",
);
```

If a `lead` prop exists, the form updates with PUT `/api/leads/:id`. Otherwise it creates with POST `/api/leads`. Email is present but optional in the current implementation; non-empty invalid email is rejected by the backend.

The detail page is a server component. It loads the lead using `loadLead(id)`, then loads notes using `apiRequest("Leads", "get", ..., /${id}/notes)`. Adding notes happens from the edit page using `NoteForm`.

The dashboard is mostly a server-rendered data page. `DashboardCharts.jsx` is a client component because Recharts needs browser-side rendering.

5. API Integration and Next.js Proxy

`frontend/api/MainApi.js` has two maps:

```
export const apiurls = {
  Dashboard: `${API_BASE_URL}/dashboard`,
  Leads: `${API_BASE_URL}/leads`,
};

export const browserApiUrls = {
  Dashboard: "/api/dashboard",
  Leads: "/api/leads",
};
```

Server-side code uses `apiurls`, which points to Express through `NEXT_PUBLIC_API_BASE_URL`. Browser-side code uses `browserApiUrls`, so the browser calls the same-origin Next proxy.

The proxy file is `frontend/app/api/[...path]/route.js`. It:

- validates allowed routes by HTTP method;
- rejects cross-origin mutations;
- parses JSON request bodies for POST and PUT;
- reads `proplead_token` from HttpOnly cookies;
- forwards the JWT to Express as `Authorization: Bearer <token>`;
- stores the login token as an HttpOnly cookie and removes it from the response body;
- deletes the cookie on logout.

Protected request flow:

```
Browser GET /api/leads
  -> Next proxy reads HttpOnly cookie
  -> Proxy calls Express GET /api/leads with Authorization header
  -> Express verifies token and role
  -> Backend returns JSON
  -> Proxy returns same status/body to browser
```

The proxy avoids exposing the backend URL and JWT directly to browser JavaScript. Development usually uses Next on port 3000 and Express on port 5000.

6. Backend Architecture, File by File

`backend/server.js` middleware order:

1. `express.json({ limit: "100kb" })`.
2. JSON object body guard for POST and PUT.
3. Public `/api/health`.
4. `authRoutes`; login is public, logout/me have route-level auth.
5. Global `authenticateToken` for remaining `/api` routes.
6. Lead, dashboard, and user routes.
7. Not-found handler.
8. Final error handler.

Actual permissions:

Route	Permission
POST /api/leads	admin, CRM
GET /api/leads	admin, CRM
GET /api/leads/:id	admin, CRM
PUT /api/leads/:id	admin, CRM
DELETE /api/leads/:id	admin only
POST/GET /api/leads/:id/notes	admin, CRM
GET /api/dashboard	admin, CRM
GET/POST /api/users	admin only
GET /api/roles	admin only

Controller excerpt:

```
const createLead = async (req, res) => {
  try {
    const lead = await leadService.createLead(req.body);
    res.status(201).json(lead);
  } catch (error) {
    handleControllerError(res, error);
  }
};
```

The controller handles HTTP status and delegates business work. The service validates and constructs entities. The repository calls database functions.

Repository excerpt:

```
createLead: (lead) => callResponse("crm_json_create_lead", values(lead), 201)
```

`callResponse` builds placeholders like `$1, $2` and passes values separately. It also checks the function name against an allowlist.

7. Lead Management - Complete CRUD

Create lead

- Method: POST
- Endpoint: `/api/leads`
- Roles: admin, CRM
- Validation: required name, phone, budget, location, property_type, source; optional email format; status default New; budget nonnegative.
- Database function: `crm_json_create_lead` -> `crm_create_lead` -> INSERT.

```
{
  "name": "Asha Verma",
  "phone": "+919876543210",
  "email": "asha@example.com",
  "budget": 7500000,
  "location": "Pune",
  "property_type": "3 BHK",
  "source": "Website"
}
```

Retrieve all leads

- Method: GET
- Endpoint: `/api/leads`
- Supports search, source, status, sortBy, sortOrder, page, pageSize.
- Function: `crm_json_list_leads`.

Retrieve one lead

- Method: GET
- Endpoint: `/api/leads/:id`
- Invalid UUID returns 400.
- Valid missing UUID returns 404.
- Function: `crm_json_get_lead`.

Update lead

- Method: PUT
- Endpoint: `/api/leads/:id`
- Same `Lead` entity validation as create.
- Function: `crm_json_update_lead`.
- SQL updates `updated_at = NOW()`.

Delete lead

- Method: DELETE
- Endpoint: `/api/leads/:id`
- Admin only.
- Function: `crm_json_delete_lead`.
- SQL uses `DELETE ... RETURNING *`.
- Related notes are removed by `ON DELETE CASCADE`.

Method	Meaning	Used here
GET	Read	list, detail, notes, dashboard
POST	Create or compatibility action	create lead/note, login, status alias
PUT	Replace/update editable lead fields	update lead
PATCH	Partial update	Not implemented; proxy returns 405
DELETE	Delete	delete lead

8. Search, Filtering, and Sorting

Listing parameters:

- `search` searches name or phone.
- `source` exact-matches lead source.
- `status` exact-matches status.
- `sortBy` must be `created_at` or `budget`.
- `sortOrder` must be `asc` or `desc`.

Frontend flow:

```
FilterForm submit
  -> navigate(nextFilters)
  -> LeadsComp useEffect
  -> apiRequest("Leads", "get", params)
```

Service allowlist:

```
if (!allowedSortFields.includes(sortBy)) {
  throw new AppError("Invalid sort field", 400);
}
if (!allowedSortOrders.includes(sortOrder)) {
  throw new AppError("Invalid sort order", 400);
}
```

Database condition:

```
WHERE (p_search IS NULL OR name ILIKE '%' || p_search || '%' OR phone LIKE '%' || p_search || '%')
  AND (p_source IS NULL OR source = p_source)
  AND (p_status IS NULL OR status = p_status)
```

Sort columns and directions are allowlisted because SQL identifiers cannot be safely parameterized like values. Indexes can help, but PostgreSQL may still choose a sequential scan for small tables or broad filters.

Index	Intended support	Limitation
idx_leads_name_lower	name search legacy function	leading wildcard may reduce use
idx_leads_phone	phone search	leading wildcard may reduce use
idx_leads_source	exact source filter	depends on selectivity
idx_leads_status	exact status filter	low-cardinality values may be less useful
idx_leads_created_at	date sort	planner-dependent
idx_leads_budget	budget sort	planner-dependent
idx_leads_created_id	stable date ordering	direction-specific
idx_leads_budget_id	stable budget ordering	direction-specific

9. Database Design

Actual tables:

Table	Purpose
leads	Main lead records
lead_notes	Notes/comments for leads
roles	Role names: admin and CRM
users	Login accounts

Entity relationship diagram:

```
roles 1 ---- many users
  roles.id = users.role_id

leads 1 ---- many lead_notes
  leads.id = lead_notes.lead_id ON DELETE CASCADE
```

`leads` columns:

Column	Type	Meaning
id	UUID primary key	Unique lead ID
name	TEXT NOT NULL	Lead name
phone	TEXT NOT NULL	Contact number
email	TEXT nullable	Optional email
budget	NUMERIC(12,2) NOT NULL	Money-like budget value
location	TEXT NOT NULL	Preferred location
property_type	TEXT NOT NULL	1 BHK, Plot, Villa, etc.
source	TEXT NOT NULL	Website, Referral, etc.
status	TEXT NOT NULL DEFAULT New	Lead stage
created_at	TIMESTAMPTZ	Creation time
updated_at	TIMESTAMPTZ	Last update time

Constraints include `leads_budget_non_negative` and `leads_status_allowed`. `lead_notes.lead_id` references `leads(id)` with cascade deletion. `users.role_id` references `roles(id)`.

Stored functions in `functions.sql` include:

- `crm_create_lead`, `crm_get_lead`, `crm_update_lead`, `crm_delete_lead`;
- `crm_list_leads` for pagination/search/filter/sort;
- `crm_add_note`, `crm_list_notes`;
- `crm_dashboard`;
- `crm_auth_user`, `crm_get_user`, `crm_list_users`, `crm_create_user`;
- `crm_json_*` wrappers that return the API success envelope.

`schema.sql` defines table structure and indexes. `functions.sql` defines application routines. The live database is only correct after those SQL files have been applied.

10. Authentication and Security

Login sequence:

```
LoginForm -> POST /api/auth/login
Next proxy -> Express authController.login
authService.login -> authRepository.getUserByEmail
verifyPassword compares supplied password with stored hash
signToken creates HS256 JWT with userId, role, iat, exp
Next proxy stores token in HttpOnly proplead_token cookie
```

JWT claims actually implemented: `userId`, `role`, `iat`, and `exp`. Expiry is one hour.

Protected API sequence:

```
Browser GET /api/leads
  -> Next proxy reads cookie
  -> forwards Authorization: Bearer token
  -> Express authenticateToken verifies signature and expiry
  -> authorizeRoles checks role
```

401 means unauthenticated or invalid token. 403 means authenticated but not permitted.

Implemented protections:

- HttpOnly cookie;
- SameSite=Lax;
- Secure cookie flag in production;
- one-hour JWT expiry;
- role middleware;
- parameterized database calls;
- input validation;
- database constraints;
- scrypt password hashing for new passwords.

Limitations:

- logout clears the browser cookie but does not revoke copied tokens;
- no refresh token rotation;
- no rate limiting or account lockout;
- no dynamic permission editor;
- no audit log.

11. Notes and Comments

`lead_notes` stores comments separately:

```
CREATE TABLE IF NOT EXISTS lead_notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
  note TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Separate storage supports one-to-many history. Create note endpoint: POST `/api/leads/:id/notes`. Retrieve notes endpoint: GET `/api/leads/:id/notes`. Notes are ordered by newest first with `ORDER BY created_at DESC, id DESC`.

Validation: lead ID must be valid, the lead must exist, note text must be non-empty after trimming, and note length must be no more than 5000 characters.

Frontend display: the lead detail page lists notes. The edit page renders `NoteForm` for adding a new note.

Example:

```
Lead: Asha Verma
Note 1: Called customer; wants 3 BHK near Hinjewadi.
Note 2: Site visit scheduled for Saturday.
```

Both notes share the same `lead_id` but have separate note IDs and timestamps.

12. Dashboard Implementation

Endpoint: GET `/api/dashboard`.

Flow:

```
dashboardRoutes -> dashboardController.getDashboard
  -> dashboardService.getDashboard
  -> dashboardRepository.getDashboardCounts
  -> crm_json_dashboard
  -> crm_dashboard
```

Metrics implemented:

- totalLeads;
- closedLeads;
- conversionRate;
- totalBudget;
- averageBudget;
- leadsBySource;
- statusDistribution;
- recentLeads.

Aggregate SQL concept:

```
SELECT COUNT(*) AS total,
       COUNT(*) FILTER (WHERE status = 'Closed') AS closed,
       COALESCE(SUM(budget), 0) AS total_budget,
       COALESCE(AVG(budget), 0) AS average_budget
FROM leads;
```

`GROUP BY` is used for source and status:

```
SELECT source, COUNT(*) FROM leads GROUP BY source;
SELECT status, COUNT(*) FROM leads GROUP BY status;
```

Conversion rate:

```
closedLeads / totalLeads * 100
```

If there are no leads, conversion rate is 0. If 5 of 20 leads are Closed, the conversion rate is 25.00%.

Frontend rendering: `DashboardComp.jsx` fetches server-side data, shows stat cards, a conversion hero, mini budget metrics, charts, and recent leads. It displays an error block if the API fails and an empty state when there are no leads.

13. User Master and Role Management

User Master is implemented as an extra feature.

Frontend:

- `app/(protected)/users/page.jsx` renders User Master.
- `UserMasterComp.jsx` calls `requireUser()` and redirects non-admin users to `/`.
- Users and roles are fetched server-side.
- `UserMasterClient.jsx` supports local search, pagination and create-user form.

Backend:

- GET `/api/users`: admin only.
- POST `/api/users`: admin only.
- GET `/api/users/:id`: admin only.
- GET `/api/roles`: admin only.

Create-user validation: name required, email required and valid, password required and at least 6 characters, roleId must be UUID, duplicate email is rejected, and selected role must exist. Passwords are hashed before storage.

Missing user features: edit user, delete user, deactivate user, reset password, and dynamic permissions.

14. Validation and Error Handling

Actual validation approach:

Rule	Implementation
Required lead strings	`Lead.js`
Optional email format	`Lead.js`
Numeric/nonnegative budget	`Lead.js` and DB CHECK
Allowed statuses	`Lead.js`, service, DB CHECK
Lead UUID	regex in `leadService.js`
User UUID	Zod utility in `utils/validation.js`
Query sorting	allowlists in `leadService.js`
Notes	`cleanNoteText` in `leadService.js`

Error flow:

```
Service throws AppError("Invalid lead ID", 400)
Controller catches error
handleControllerError sends apiResponse(400, "Invalid lead ID")
Client receives ok:false JSON
```

Status codes:

Code	Meaning in this app
400	Invalid input
401	Missing/invalid/expired token or invalid login
403	Authenticated but not allowed
404	Missing route or resource
409	Not currently used as a standard response
500	Unexpected failure; message hidden as Internal server error

The backend dependency list still includes Zod. It is not used for lead schemas, but it is used by `utils/validation.js` for user UUID validation.

15. Environment and Local Setup

Prerequisites:

- Node.js 20.9+;
- PostgreSQL 13+ or Supabase PostgreSQL;
- npm;
- backend and frontend env files.

Safe env examples:

- `backend/.env.example`;
- `frontend/.env.example`.

Backend example:

```
DATABASE_URL=postgresql://USER:PASSWORD@HOST:PORT/DATABASE
JWT_SECRET=replace-with-a-long-random-secret
PORT=5000
```

Frontend example:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:5000/api
```

Setup commands:

```
npm ci --prefix backend
npm ci --prefix frontend
npm run db:setup --prefix backend
npm run seed-demo --prefix backend
npm run dev:backend
npm run dev:frontend
```

Health check:

```
curl http://localhost:5000/api/health
```

Troubleshooting: check `JWT_SECRET`, `DATABASE_URL`, `NEXT_PUBLIC_API_BASE_URL`, and whether `db:setup` has been run. The app startup does not apply migrations automatically.

16. Testing and Verification

Evidence available from recent work:

- `npm run lint` passed.
- `npm run build:frontend` passed.
- Env ignore behavior verified: real env files ignored and `.env.example` files trackable.
- Functional workflow verified through Next proxy, Express and PostgreSQL: create, persistence, list, name search, phone search, source/status filters, date/budget sort, detail fetch/page load, note create/read, status update, dashboard update, CRM permission denial, admin delete, deleted lead 404 and dashboard baseline restore.

Not claimed here:

- final manual browser click acceptance inspection;
- production security hardening;
- dynamic permissions;
- refresh-token revocation.

Safe test pattern:

```
Create uniquely named test lead.
Store the returned ID.
Run reads/updates/notes/dashboard checks.
Before cleanup, verify the ID still belongs to that test lead.
Delete only that ID.
```

17. End-to-End Data Flow Examples

Login:

```
LoginForm.jsx -> apiRequest("Login", "post")
Next proxy /api/auth/login
Express authController.login
authService.login
authRepository.getUserByEmail
crm_json_auth_user
verifyPassword
signToken
crm_json_session
Proxy stores HttpOnly cookie
```

Create lead:

```
LeadForm.jsx -> POST /api/leads
leadRoutes.js -> leadController.createLead
leadService.createLead -> new Lead(leadData)
leadRepository.createLead -> crm_json_create_lead
PostgreSQL INSERT INTO leads
```

List/search leads:

```
LeadsComp state -> GET /api/leads?search=...
leadService.validateLeadQueryParams
leadRepository.getAllLeads
crm_json_list_leads
crm_list_leads applies WHERE, ORDER BY, LIMIT/OFFSET
```

Retrieve detail:

```
/leads/[id]/page.jsx -> loadLead(id)
loadLead validates UUID and calls GET /api/leads/:id
Page also fetches GET /api/leads/:id/notes
```

Update status:

```
LeadForm edit submits PUT /api/leads/:id with status
Lead entity validates status
crm_update_lead updates status and updated_at
```

Add/retrieve note:

```
NoteForm -> POST /api/leads/:id/notes
leadService verifies lead exists
crm_add_note inserts lead_notes row
Detail page GET /api/leads/:id/notes retrieves ordered notes
```

Delete lead:

```
LeadActions delete -> DELETE /api/leads/:id
admin role required
crm_delete_lead deletes lead
PostgreSQL cascades note deletion
```

CRM blocked:

```
CRM user -> GET /api/users
authenticateToken succeeds
authorizeRoles("admin") fails
Response: 403 Access denied
```

18. Interview Preparation

Two-minute explanation:

PropLead CRM is a full-stack real estate lead management system. It uses Next.js, React and Tailwind for the frontend, Express for REST APIs, and PostgreSQL/Supabase for persistence. Users log in as admin or CRM. They can capture leads, search/filter/sort, view detail pages, add notes, update statuses and view dashboard metrics. The backend is layered into routes, controllers, services, repositories and PostgreSQL functions. Authentication uses password hashing, JWTs and an HttpOnly cookie managed by a Next API proxy.

Important design decisions:

- Layered backend keeps HTTP, business logic and database access separate.
- PostgreSQL functions centralize database operations.
- The Next proxy keeps JWTs out of browser JavaScript.
- Role middleware protects admin-only actions.
- Database constraints back up application validation.
- Allowlisted sorting protects against SQL injection.

Interview Q&A:

1. What problem does PropLead solve? It organizes real estate enquiries and follow-ups.
2. Why Next.js? It provides routing, server components, layouts and API proxy.
3. Why Express? It provides a clear REST API and middleware pipeline.
4. Why PostgreSQL? The data is relational and needs constraints and relationships.
5. What is the backend flow? Route -> controller -> service -> repository -> database function.
6. What does a controller do? It handles HTTP input/output.
7. What does a service do? It applies business rules and validation.
8. What does a repository do? It accesses data through database functions.
9. Why not put SQL in controllers? That couples HTTP and storage concerns.
10. How is SQL injection prevented? Values are parameterized and sort fields are allowlisted.
11. Why allowlist sort fields? SQL identifiers cannot be safely parameterized like values.
12. How is auth implemented? Email/password login, password hash verification, JWT, HttpOnly cookie.
13. What is in the JWT? userId, role, iat and exp.
14. Why HttpOnly cookie? Browser JavaScript cannot read it.
15. Difference between 401 and 403? 401 is unauthenticated; 403 is unauthorized.
16. How are roles enforced? `authorizeRoles` checks `req.user.role`.
17. Is email required? No, it is optional but validated when provided.
18. How is budget validated? Numeric, finite, nonnegative, max checked, plus DB CHECK.
19. How are statuses validated? Entity/service checks and DB CHECK.
20. Why UUIDs? They avoid predictable IDs and work well across systems.
21. What does ON DELETE CASCADE do? Deletes notes automatically when the lead is deleted.
22. What dashboard SQL is used? COUNT, FILTER, SUM, AVG and GROUP BY.
23. What is conversion rate? Closed leads divided by total leads times 100.
24. What are limitations? No refresh token, no rate limit, no final browser acceptance here.
25. How would you extend it? Add audit logs, assignment, imports, exports and Playwright tests.

Likely follow-ups include middleware order, proxy allowlists, stored functions, indexes, validation, JWT expiry, cookie security and role permissions.

19. Final Implementation Summary

Stack: Next.js App Router, React, Tailwind CSS, Express.js, PostgreSQL/Supabase, pg, JWT, scrypt password hashing, Recharts, SweetAlert2 and lucide-react.

Main files:

Area	Files
Express entry	`backend/server.js`
Lead backend	`leadRoutes.js`, `leadController.js`, `leadService.js`, `leadRepository.js`, `Lead.js`
Auth backend	`authRoutes.js`, `authService.js`, `authRepository.js`, `authMiddleware.js`
Database	`db.js`, `routines.js`, `schema.sql`, `functions.sql`
Frontend API	`MainApi.js`, `apiRequest.js`, `app/api/[...path]/route.js`
Auth frontend	`lib/auth.js`, login page, protected layout
Dashboard UI	`DashboardComp.jsx`, `DashboardCharts.jsx`, `SyncButton.jsx`
Lead UI	`LeadsComp.jsx`, `LeadForm.jsx`, `LeadControls.jsx`, `LeadActions.jsx`
User Master	`UserMasterComp.jsx`, `UserMasterClient.jsx`

API summary:

Method	Endpoint	Purpose
POST	/api/auth/login	Login
GET	/api/auth/me	Current user
POST	/api/auth/logout	Logout
POST	/api/leads	Create lead
GET	/api/leads	List/search/filter/sort
GET	/api/leads/:id	Lead detail
PUT	/api/leads/:id	Update lead
DELETE	/api/leads/:id	Delete lead
POST	/api/leads/:id/notes	Add note
GET	/api/leads/:id/notes	List notes
GET	/api/dashboard	Dashboard
GET/POST	/api/users	User Master
GET	/api/roles	Roles

Implemented features: lead CRUD, search, filters, sorting, details, notes, status update, dashboard, auth, RBAC, User Master, setup docs and env examples.

Outstanding gaps: no final browser acceptance inspection in this document, no refresh-token revocation, no rate limiting, no audit log, no dynamic permission management and no automated browser test suite.

Study order:

1. `schema.sql`
2. `functions.sql`
3. Lead route/controller/service/repository/model

4. Auth middleware and JWT helper
5. Next proxy and API helper
6. Protected layout and AppShell
7. LeadsComp and LeadForm
8. DashboardComp and dashboard SQL
9. Practice one request end to end

Glossary:

Term	Meaning
API	Contract for clients to request data/actions
Controller	Backend HTTP request/response layer
Service	Business rules layer
Repository	Database access layer
JWT	Signed token carrying claims
HttpOnly cookie	Cookie inaccessible to browser JavaScript
RBAC	Role-based access control
UUID	Globally unique identifier
Foreign key	Database relationship constraint
Index	Structure that can speed reads with write/storage cost
Stored function	PostgreSQL routine called by app code